

Diffusion Forcing: Next-token Prediction Meets Full-Sequence Diffusion

Boyuan Chen, Diego Marti Monso, Yilun Du, Max Simchowitz, Russ Tedrake, Vincent Sitzmann
(MIT CSAIL, NeurIPS 2024)

Pengxi Liu

August 20, 2026

Motivation

Probabilistic sequence modeling powers modern ML: language, video, decision-making.

Today, there are two dominant paradigms:

Motivation

Probabilistic sequence modeling powers modern ML: language, video, decision-making.

Today, there are two dominant paradigms:

- **Next-token prediction:** GPT-style autoregressive models.
 - **No guidance mechanism:** cannot guide sampling toward an objective.
 - **Unstable on continuous data:** frame-to-frame errors accumulate; auto-regressive video diverges past the training horizon.

Motivation

Probabilistic sequence modeling powers modern ML: language, video, decision-making.

Today, there are two dominant paradigms:

- **Next-token prediction:** GPT-style autoregressive models.
 - **No guidance mechanism:** cannot guide sampling toward an objective.
 - **Unstable on continuous data:** frame-to-frame errors accumulate; auto-regressive video diverges past the training horizon.
- **Full-sequence diffusion:** Sora-style video diffusion.
 -  Supports guidance; great on continuous signals (video).
 -  **Non-causal, unmasked** architecture.
 -  **Fixed-length only:** no variable-length generation.

Motivation

Probabilistic sequence modeling powers modern ML: language, video, decision-making.

Today, there are two dominant paradigms:

- **Next-token prediction:** GPT-style autoregressive models.
 - **No guidance mechanism:** cannot guide sampling toward an objective.
 - **Unstable on continuous data:** frame-to-frame errors accumulate; auto-regressive video diverges past the training horizon.
- **Full-sequence diffusion:** Sora-style video diffusion.
 - ✓ Supports guidance; great on continuous signals (video).
 - ✗ **Non-causal, unmasked** architecture.
 - ✗ **Fixed-length only:** no variable-length generation.

Unifying both might be a better option...

Preliminaries: Diffusion Models

- **Forward process:** $q(x^k | x^{k-1}) = \mathcal{N}(x^k; \sqrt{1 - \beta_k} x^{k-1}, \beta_k I)$.

Closed-form marginal, with $\bar{\alpha}_k = \prod_{j=1}^k (1 - \beta_j)$: $x^k = \sqrt{\bar{\alpha}_k} x^0 + \sqrt{1 - \bar{\alpha}_k} \epsilon$, $\epsilon \sim \mathcal{N}(0, I)$.
 $p(x^k | x^0) = \mathcal{N}(x^k; \sqrt{\bar{\alpha}_k} x^0, (1 - \bar{\alpha}_k) I)$. After K steps: $x^K \approx \mathcal{N}(0, I)$.

- **Reverse process:** $p_\theta(x^{k-1} | x^k) = \mathcal{N}(x^{k-1}; \mu_\theta(x^k, k), \gamma_k I)$.

- **Training loss:** $L(\theta) = \mathbb{E}_{k, x^0, \epsilon} \|\epsilon - \epsilon_\theta(x^k, k)\|^2$.

- **Guidance:**

- Score $\leftrightarrow$ noise: $\nabla_{x^k} \log p(x^k) = -\frac{x^k - \sqrt{\bar{\alpha}_k} x^0}{1 - \bar{\alpha}_k} = -\frac{\epsilon}{\sqrt{1 - \bar{\alpha}_k}}$.
- $\nabla_{x^k} \log p(x^k | y) = \nabla_{x^k} \log p(x^k) + \nabla_{x^k} \log p(y | x^k) \leftrightarrow$
 $\epsilon_\theta(x^k, k; y) = \epsilon_\theta(x^k, k) - \sqrt{1 - \bar{\alpha}_k} \nabla_{x^k} \log p(y | x^k)$
- Intuition for the coefficient $\sqrt{1 - \bar{\alpha}_k}$:
 - $k \rightarrow 0$: $\sqrt{1 - \bar{\alpha}_k} \rightarrow 0$, clean data, tiny classifier guidance suffices.
 - $k \rightarrow K$: $\sqrt{1 - \bar{\alpha}_k} \rightarrow 1$, pure noise, full classifier gradient needed to guide sampling

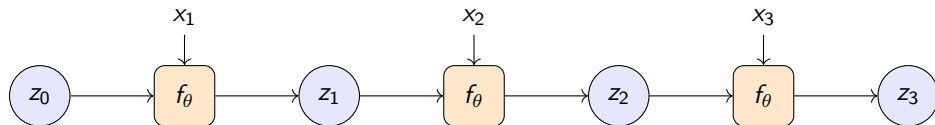
Preliminaries: Bayesian Filtering and RNN

Bayesian filtering: track a hidden state z_t from noisy observations x_t over time.

Two operations the model keeps running:

- **Prior:** roll forward without any new info $p(z_{t+1} | z_t)$.
- **Posterior:** a new observation arrives, update the belief $p(z_{t+1} | z_t, x_{t+1})$

RNN: process a sequence by maintaining a hidden state that summarizes the past.

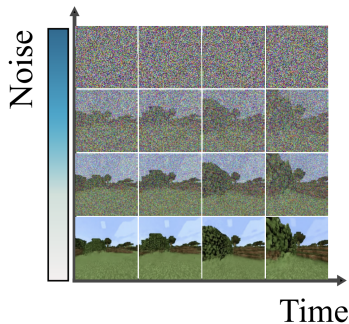


- Hidden state z_t = belief over the past
- $f_\theta(z_{t-1}, x_t)$ = posterior update (merge belief with new obs)
- $f_\theta(z_{t-1}, \text{no obs})$ = prior prediction (roll forward)

Method: Noising as Partial Masking

Noising tokens is a form of partial masking.

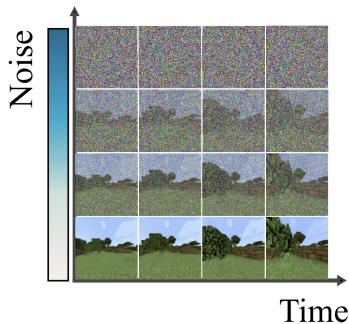
- Noise level $k = 0$: token is **unmasked** (clean, informative)
- Noise level $k = K$: token is **fully masked** (pure noise, uninformative)
- Intermediate k : token is **partially masked**



Method: Noising as Partial Masking

Noising tokens is a form of partial masking.

- Noise level $k = 0$: token is **unmasked** (clean, informative)
- Noise level $k = K$: token is **fully masked** (pure noise, uninformative)
- Intermediate k : token is **partially masked**

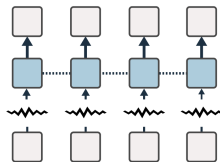


So why not let each token have its own noise level? → Train a single model to “unmask” any collection of variably-noised tokens.

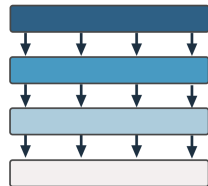
Method: Diffusion Forcing

Diffusion Forcing is a framework for training and sampling arbitrary sequence lengths of noisy tokens $(x_t^{k_t})_{1 \leq t \leq T}$, where critically, the noise level k_t of each token can vary by time step.

Full-Seq. Diffusion

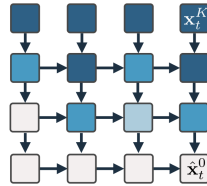
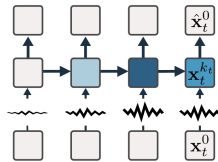


• Training



• Sampling

Diffusion Forcing



Method: Causal Diffusion Forcing (RNN Implement)

For time series data, instantiate Diffusion Forcing with causal architectures noted as **Causal Diffusion Forcing**. Each time step has two components:

1. Dynamics model:

$$z_t \sim p_{\theta}(z_t \mid z_{t-1}, x_t^{k_t}, k_t).$$

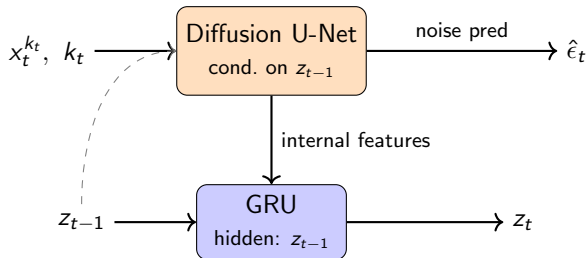
Take prior belief z_{t-1} , current noisy observation $x_t^{k_t}$, and its noise level k_t ; produce updated belief z_t .

2. Prediction head: given belief, output ϵ -prediction:

$$\epsilon_{\theta}(z_{t-1}, x_t^{k_t}, k_t).$$

Method: Causal Diffusion Forcing (RNN Implement)

he RNN unit has two sub-networks chained together.



- **U-Net** output $\hat{\epsilon}_t \rightarrow$ supervised by diffusion loss $\|\hat{\epsilon}_t - \epsilon_t\|^2$
- **Gated Recurrent Unit (GRU)** updates belief $z_t \rightarrow$ emerge naturally because the next step's U-Net is conditioned on it

Method: Diffusion Forcing (Training)

Algorithm 1 Diffusion Forcing Training

```
1: loop
2:   Sample trajectory  $(x_1, \dots, x_T)$ 
3:   for  $t = 1, \dots, T$  do
4:     Sample independent noise level  $k_t \in \{0, 1, \dots, K\}$ 
5:      $x_t^{k_t} \leftarrow \text{ForwardDiffuse}(x_t, k_t)$ 
6:     Compute ground-truth noise  $\epsilon_t$ 
7:     Update latent  $z_t \sim p_\theta(z_t \mid z_{t-1}, x_t^{k_t}, k_t)$ 
8:     Predict  $\hat{\epsilon}_t = \epsilon_\theta(z_{t-1}, x_t^{k_t}, k_t)$ 
9:   end for
10:   $L = \text{MSE}([\hat{\epsilon}_1, \dots, \hat{\epsilon}_T], [\epsilon_1, \dots, \epsilon_T])$ 
11:  Backprop and update  $\theta$ 
12: end loop
```

Method: Diffusion Forcing (Training)

Training objective:

$$\mathbb{E}_{k_{1:T}, x_{1:T}, \epsilon_{1:T}} \sum_{t=1}^T \left\| \epsilon_t - \epsilon_{\theta}(z_{t-1}, x_t^{k_t}, k_t) \right\|^2$$

- $k_{1:T}$ sampled i.i.d. uniform from $\{0, \dots, K\}^T$.
- Noise is generated per-token independently.

Method: Diffusion Forcing (Sampling)

A single trained Diffusion Forcing model can sample in many ways, specified by a 2D **scheduling matrix** $\mathcal{K} \in [K]^{M \times T}$:

- Columns = time steps t
- Rows m = denoising iterations
- $\mathcal{K}_{m,t}$ = desired noise level of token t at iteration m

Procedure:

- 1 Initialize all tokens $x_{1:T}$ as white noise (noise level K)
- 2 For $m = M-1, M-2, \dots, 0$ (iterate *top-down* through $\mathcal{K}$):
 - For $t = 1, 2, \dots, T$ (iterate *left-to-right*):
 - Update latent: $z_t \leftarrow f_\theta(z_{t-1}, x_t, \mathcal{K}_{m+1,t})$
 - Take one denoising step to bring x_t from level $\mathcal{K}_{m+1,t}$ to $\mathcal{K}_{m,t}$
 - (Optional) apply guidance to the partially denoised $x_{1:T}$
- 3 At $m = 0$: all tokens clean, sequence done

Method: Diffusion Forcing (Sampling)

- Full-sequence diffusion

$$\mathcal{K} = \begin{pmatrix} K & K & K & K \\ K-1 & K-1 & K-1 & K-1 \\ \vdots & \vdots & \vdots & \vdots \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

- Pyramid (causal structure):

$$\mathcal{K}_{\text{pyramid}} = \begin{pmatrix} K & K & K & K \\ K-1 & K & K & K \\ K-2 & K-1 & K & K \\ \vdots & \vdots & \vdots & \vdots \\ 0 & 1 & 2 & 3 \\ 0 & 0 & 1 & 2 \\ \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Method: New Capabilities in Sequence Generation

- **Keeping the future uncertain:**

Start: $[x_1^K, x_2^K, x_3^K]$

Then: $[x_1^0, x_2^{K/2}, x_3^K]$

Then: $[x_1^0, x_2^0, x_3^{K/2}]$

Finally: $[x_1^0, x_2^0, x_3^0]$

- Pyramid-shaped $\mathcal{K}$ matrix.
 - Noise level $\approx$ uncertainty.
 - Used in planning: commit to near actions, keep distant futures flexible.
- **Long-horizon guidance.** Due to the dependency of future tokens on the past, guidance gradients from future tokens can propagate backwards in time

$$x_{1:T} \leftarrow x_{1:T} + \eta \nabla_{x_{1:T}} \log c(x_{1:T})$$

- Gradient of future reward flows backward in time.
 - Past tokens get guided by *long-horizon* future reward.
 - Still respect causality (future depends on past).

Method: Monte Carlo Guidance

Assume we want to maximize expected future reward (common for RL):

$$c(x_t^k) = \mathbb{E} \left[\sum_{t' > t} r(x_{t'}^{k_{t'}}) \mid x_t^k \right]$$

Monte Carlo Guidance:

- Draw multiple samples of future tokens $x_{t+1:T}$.
- Average guidance gradients across samples.

Why only DF can do this: full-sequence diffusion denoises every token at the same level, thus no randomness left for sampling futures. DF's independent noise lets us keep future tokens noisy while near future is clean.

Method: Sampling with Guidance

Algorithm 2 DF Sampling with Guidance

- 1: **Input:** Model θ , scheduling matrix $\mathcal{K}$, initial latent $\mathbf{z}_0$, guidance cost $c(\cdot)$.
 - 2: **Initialize** $\mathbf{x}_1, \dots, \mathbf{x}_T \sim \mathcal{N}(0, \sigma_K^2 I)$.
 - 3: **for** row $m = M - 1, \dots, 0$ **do**
 - 4: **for** $t = 1, \dots, T$ **do**
 - 5: $\mathbf{z}_t^{\text{new}} \sim p_\theta(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_t, \mathcal{K}_{m+1,t})$.
 - 6: $k \leftarrow \mathcal{K}_{m,t}$, $\mathbf{w} \sim \mathcal{N}(0, \mathbf{I})$.
 - 7: $\mathbf{x}_t^{\text{new}} \leftarrow \frac{1}{\sqrt{\alpha_k}} \left(\mathbf{x}_t - \frac{1-\alpha_k}{\sqrt{1-\alpha_k}} \epsilon_\theta(\mathbf{z}_t^{\text{new}}, \mathbf{x}_t, k) \right) + \sigma_k \mathbf{w}$
 - 8: **Update** $\mathbf{z}_t \leftarrow \mathbf{z}_t^{\text{new}}$.
 - 9: **end for**
 - 10: $\mathbf{x}_{1:H} \leftarrow \text{AddGuidance}(\mathbf{x}_{1:H}^{\text{new}}, \nabla_{\mathbf{x}} \log c(\mathbf{x}_{1:H}^{\text{new}}))$
 - 11: **end for**
 - 12: **Return** $\mathbf{x}_{1:T}$.
-

Method: Decision Making Framework

Setup. MDP with dynamics $p(s_{t+1}|s_t, a_t)$, observations $p(o_t|s_t)$, rewards $p(r_t|s_t, a_t)$.

Tokens: $x_t = [a_t, r_t, o_{t+1}]$. Trajectory = sequence $x_{1:T}$.

At execution time t :

- ① Past clean tokens summarized in latent z_{t-1}
- ② Sample a plan $\hat{x}_{t:t+H}$ via scheduling matrix $\mathcal{K}$
- ③ Execute planned action $\hat{a}_t$; environment returns r_t, o_{t+1}
- ④ Update latent: $z_t \sim p_\theta(z_t | z_{t-1}, x_t, 0)$

Lookahead H can be **tuned at sampling time**:

- Small $H \rightarrow$ low-latency *policy*
- Large $H \rightarrow$ long-horizon *planning*

Experiments: Overview

- ➊ **Video prediction:** Minecraft, DMLab (consistency)
- ➋ **Planning:** D4RL Maze2D (MCG, flexible horizon)
- ➌ **Robot imitation learning:** Fruit-swap task (memory, robustness)
- ➍ **Time series forecasting:** 6 GluonTS datasets (sanity check)

Experiment: Video Prediction

Baselines:

- **Teacher forcing** next-frame diffusion.
- **Full-sequence diffusion**.

Datasets:

- **Minecraft**: gameplay, 128×128 , trained on 72-frame subsequences
- **DMLab**: navigation, 64×64 , trained on 36-frame subsequences

Result: <https://www.boyuan.space/diffusion-forcing/>

- Teacher forcing diverges quickly.
- Full-sequence diffusion suffers from frame-to-frame discontinuity.

Experiment: Planning

Baselines:

- SOTA offline RL methods.
- Diffuser, a diffusion planning framework.

Dataset:

- **D4RL**: a standard offline RL benchmark with a set of 2D maze environments. An agent is tasked with reaching a designated goal position starting from a random starting position.

Result:

Environment		MPPI	CQL	IQL	Diffuser*	Diffuser w/ diffused action	Ours wo/ MCG	Ours
Maze2D	U-Maze	33.2	5.7	47.4	113.9 ± 3.1	6.3 ± 2.1	110.1 ± 3.9	116.7 ± 2.0
Maze2D	Medium	10.2	5.0	34.9	121.5 ± 2.7	13.5 ± 2.3	136.1 ± 10.2	149.4 ± 7.5
Maze2D	Large	5.1	12.5	58.6	123.0 ± 6.4	6.3 ± 2.1	142.8 ± 5.6	159.0 ± 2.7
Single-task Average		16.2	7.7	47.0	119.5	8.7	129.67	141.7
Multi2D	U-Maze	41.2	-	24.8	128.9 ± 1.8	32.8 ± 1.7	107.7 ± 4.9	119.1 ± 4.0
Multi2D	Medium	15.4	-	12.1	127.2 ± 3.4	22.0 ± 2.7	145.6 ± 6.5	152.3 ± 9.9
Multi2D	Large	8.0	-	13.9	132.1 ± 5.8	6.9 ± 1.7	129.8 ± 1.5	167.1 ± 2.7
Multi-task Average		21.5	-	16.9	129.4	20.6	127.7	146.2

Experiment: Robotics

Task: Robot arm swaps apple and orange.

Non-Markovian: when one fruit is in the middle slot, observation alone is ambiguous, so must remember initial configuration.

Results: <https://www.boyuan.space/diffusion-forcing/>

- Diffusion Forcing: **80% success**
- Diffusion Policy (no memory): **fails**

Experiment: Time Series Prediction

Task: Multivariate Probabilistic Time Series Forecasting

$$q(x_{t_0:T} \mid x_{1:t_0-1}), \quad x_t \in \mathbb{R}^D$$

Dataset	Content	Dim.	Freq.
Exchange	Currency rates vs. USD	8	business day
Solar	Alabama solar plants output	137	hour
Electricity	Portuguese household usage	370	hour
Traffic	SF Bay Area road occupancy	963	hour
Taxi	NYC taxi rides per region	1,214	30-min
Wikipedia	Daily page views	2,000	day

Metric: CRPS_{sum} : measures how well the predicted CDF matches the true value (lower = better).

Experiment: Time Series Prediction

Method	Exchange	Solar	Electricity	Traffic	Taxi	Wikipedia
VES [36]	0.005 ± 0.000	0.900 ± 0.003	0.880 ± 0.004	0.350 ± 0.002	-	-
VAR [45]	0.005 ± 0.000	0.830 ± 0.006	0.039 ± 0.001	0.290 ± 0.001	-	-
VAR-Lasso [45]	0.012 ± 0.000	0.510 ± 0.006	0.025 ± 0.000	0.150 ± 0.002	-	3.100 ± 0.004
GARCH [62]	0.023 ± 0.000	0.880 ± 0.002	0.190 ± 0.001	0.370 ± 0.001	-	-
DeepAR [55]	-	0.336 ± 0.014	0.023 ± 0.001	0.055 ± 0.003	-	0.127 ± 0.042
LSTM-Copula [54]	0.007 ± 0.000	0.319 ± 0.011	0.064 ± 0.008	0.103 ± 0.006	0.326 ± 0.007	0.241 ± 0.033
GP-Copula [54]	0.007 ± 0.000	0.337 ± 0.024	0.025 ± 0.002	0.078 ± 0.002	0.208 ± 0.183	0.086 ± 0.004
KVAE [41]	0.014 ± 0.002	0.340 ± 0.025	0.051 ± 0.019	0.100 ± 0.005	-	0.095 ± 0.012
NKF [14]	-	0.320 ± 0.020	0.016 ± 0.001	0.100 ± 0.002	-	0.071 ± 0.002
Transformer-MAF [51]	0.005 ± 0.003	0.301 ± 0.014	0.021 ± 0.000	0.056 ± 0.001	0.179 ± 0.002	0.063 ± 0.003
TimeGrad [50]	0.006 ± 0.001	0.287 ± 0.020	0.021 ± 0.001	0.044 ± 0.006	0.114 ± 0.020	0.049 ± 0.002
ScoreGrad sub-VP SDE [68]	0.006 ± 0.001	0.256 ± 0.015	0.019 ± 0.001	0.041 ± 0.004	0.101 ± 0.004	0.043 ± 0.002
Ours	0.003 ± 0.001	0.289 ± 0.002	0.023 ± 0.001	0.040 ± 0.004	0.075 ± 0.002	0.085 ± 0.007

Accommodation

Is it worth reading? **Yes!**

- Learn a unified perspective of next-token prediction and full-sequence diffusion.
- Learn how independent per-token noise unlocks capabilities of variable horizon, causal uncertainty, guidance.

Is it worth implementing? **Yes, directly aligned with my research.**

- Diffusion language models fit DF naturally, and also EHR trajectories.
- Check their following paper where they replace the RNN with transformer:
<https://www.boyuan.space/history-guidance/>.